Data Engineering for AI Applications

Foundation Session 4

SESSION ROADMAP

Two Parts, One Learning Outcome

Part 1

Data Foundations & Cleaning

1. Structured, semi-structured & unstructured data
2. File formats: CSV, JSON, PDF, DOCX, HTML
3. ETL fundamentals, hands-on demo
4. Data cleaning pipelines, hands-on demo

Part 2

Text, Datasets & Responsible Data Practice

1. Text preprocessing & OCR awareness
2. Annotation fundamentals & dataset quality
3. Dataset versioning & SQL basics
4. Data privacy & governance fundamentals

Learning Outcome: Understand how data quality, structure, and pipelines affect downstream AI system performance.

Why Data Engineering Matters for AI

Every AI and GenAI system is only as good as the data behind it.

Messy, incomplete, or poorly structured data leads directly to unreliable models — wrong predictions, biased outcomes, and hallucinating chatbots.

Data engineering is the discipline that makes AI systems trustworthy — by ensuring the right data, in the right shape, reaches the model.

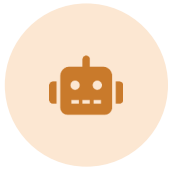
"Garbage In, Garbage Out"

A model trained or prompted with flawed data will confidently produce flawed answers — no algorithm can fix that downstream.

- Wrong predictions
- Biased outcomes
- Hallucinated answers

Real-World AI Failures Caused by Bad Data

These are not hypothetical — poor data practices are behind many well-known AI failures.



Hallucinating Chatbot

Sparse, outdated context data led a support bot to invent policies that never existed.

Biased Hiring Model

Historical hiring data skewed toward one group taught the model to penalize other candidates.

Broken Pipeline

An unversioned schema change silently broke a downstream training pipeline for weeks.

Faulty Financial Model

Missing transaction values silently became zeros, understating risk in a credit model.

Structured vs Unstructured Data

AI systems must consume many different data shapes — each needs a different processing approach.

STRUCTURED DATA

Organized in rows & columns — easy for machines to query directly

CSV / Tables

Rows of records — sales logs, user tables, transaction history

Relational DB

Structured storage queried directly using SQL

UNSTRUCTURED DATA

No fixed format — needs chunking & embedding before AI can use it

JSON

Semi-structured API & config data

PDFs / Documents

Reports, manuals, policy docs

Images

Scans, photos, diagrams

Audio

Calls, voice notes, transcripts

Semi-Structured Data: The Middle Ground

Not a rigid table, but not a formless blob either.

Semi-Structured Data

Semi-structured data has organizational markers — tags, keys, or labels — but no fixed schema like a database table, allowing flexible, nested structure.

- Examples: JSON, XML, YAML, log files, NoSQL documents
- Common source: APIs, config files, application logs, NoSQL databases
- AI systems need nested parsing logic — fields can be optional or deeply nested

EXAMPLE:

```
{
  "customer": "A. Rao",
  "orders": [
    {"item": "Laptop", "amount": 52000},
    {"item": "Mouse", "amount": 800}
  ]
}
```

File Formats: CSV

The simplest structured format — rows and columns separated by commas.

Comma-Separated Values

CSV stores tabular data as plain text, with each line representing a row and commas separating column values.

- Extremely portable — opens in Excel, pandas, or any text editor
- No native support for nested data or data types (everything is text)
- Common use: exported reports, transaction logs, tabular datasets

EXAMPLE:

```
id,name,amount,date
1,A. Rao,1200,2026-01-04
2,B. Khan,980,2026-01-05
```

File Formats: JSON

A flexible, nested format built from key-value pairs.

JavaScript Object Notation

JSON represents data as nested key-value pairs and lists — the standard format for APIs and configuration files.

- Supports nested objects and arrays — unlike flat CSV
- Human-readable and widely supported across programming languages
- Common use: API responses, config files, NoSQL document stores

EXAMPLE

```
{
  "name": "A. Rao",
  "city": "Mumbai",
  "active": true
}
```

File Formats: PDF

Fixed-layout documents built for human reading, not machine parsing.

Portable Document Format

PDFs preserve exact visual layout across devices — which is precisely what makes them hard for AI systems to parse reliably.

- Text, tables, and images can all be embedded in one file
- Extraction libraries can scramble reading order in multi-column layouts
- Scanned PDFs contain no text at all — they need OCR (covered later)

PARSING CHALLENGE

A two-column PDF report may extract as jumbled, interleaved text unless a layout-aware parser is used.

File Formats: DOCX

Microsoft Word's structured XML-based document format.

Word Documents

DOCX files are actually a zipped bundle of XML files — giving programmatic access to text, styles, tables, and comments.

- Structure (headings, styles, tables) is preserved and extractable
- Common use: contracts, letters, policy drafts, reports
- Libraries like python-docx can read and edit content directly

PARSING NOTE

Unlike PDF, DOCX retains a structured XML tree — making heading levels and tables far easier to extract accurately.

File Formats: HTML

The markup language behind every web page.

HyperText Markup Language

HTML structures content using nested tags — AI pipelines that scrape web content must parse this tag structure to extract meaningful text.

- Content is wrapped in tags: <p>, <table>, , , and more
- Scraping requires filtering out navigation, ads, and scripts
- Common use: web scraping, documentation ingestion, email bodies

EXAMPLE:

```
<article>
  <h1>Billing FAQ</h1>
  <p>How do I reset my password?</p>
</article>
```

Choosing the Right Format for Your Use Case

Three tiers of structure — each suited to different AI workflows.

Structured

- CSV, SQL tables
- Best for: analytics, dashboards, direct querying
- Easiest to validate and clean

Semi-Structured

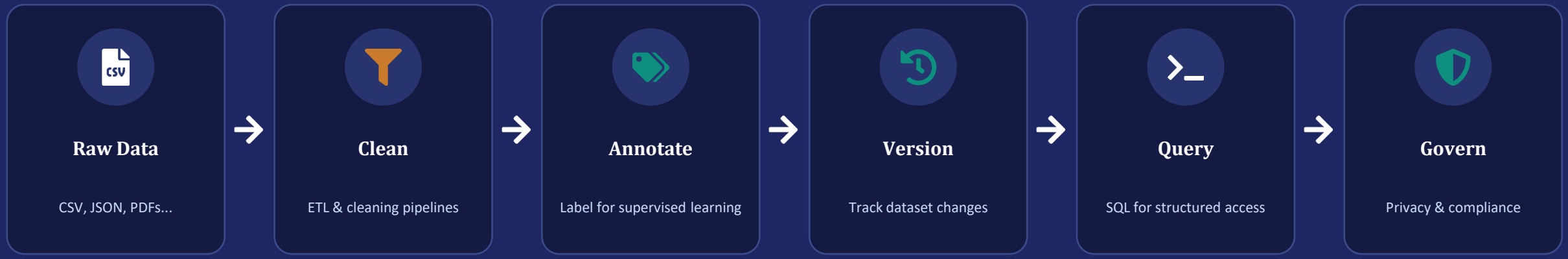
- JSON, XML, logs
- Best for: APIs, configs, NoSQL pipelines
- Needs schema-aware parsing

Unstructured

- PDF, DOCX, HTML, images, audio
- Best for: documents, RAG, OCR pipelines
- Needs extraction before any AI use

THE BIG PICTURE

The Data Engineering Lifecycle for AI



Every stage in this pipeline is covered hands-on in today's session.

PART 1 · ETL

ETL Fundamentals: Extract

Stage 1 of 3 — pulling raw data from its source, exactly as it exists.

Extract

Extraction pulls data out of its original source system — without modifying it — so it can be processed further downstream.

- Sources include databases, flat files, APIs, logs, and event streams
- Data is pulled as-is: messy, inconsistent, or incomplete
- Extraction jobs can run on a schedule or be triggered by events

EXAMPLE

A nightly job connects to a MySQL orders database and extracts all new order records created in the last 24 hours into a staging file.

ETL Fundamentals: Transform

Stage 2 of 3 — cleaning and reshaping data into a usable form.

Transform

Transformation applies rules to clean, standardize, and reshape extracted data so it is consistent and analysis-ready.

- Fix data types and standardize formats (e.g., dates, currencies)
- Remove duplicates and handle missing values
- Join, filter, and aggregate fields to match business logic

EXAMPLE

Raw order dates in three different formats are standardized to YYYY-MM-DD, and rows with a negative amount are flagged for review.

ETL Fundamentals: Load

Stage 3 of 3 — writing processed data to its destination.

Load

Loading writes the cleaned, transformed data into its final destination — ready for querying, reporting, or feeding AI systems.

- Destinations include data warehouses, databases, or file stores
- Loads can fully replace old data or append incrementally
- Well-loaded data is what SQL and downstream AI pipelines query

EXAMPLE

The cleaned order records are loaded into a PostgreSQL analytics table, replacing yesterday's snapshot for the sales dashboard.

Batch vs Streaming ETL

Two ways to move data through a pipeline — choose based on how fresh the data needs to be.

Batch ETL

- Processes data in scheduled chunks (e.g., nightly, hourly)
- Simpler to build, test, and monitor
- Higher latency — data can be hours old
- Good fit: daily sales reports, monthly billing

Streaming ETL

- Processes data continuously, record by record
- Needs infrastructure like Kafka or Kinesis
- Near real-time — seconds of latency
- Good fit: fraud detection, live dashboards

Common ETL Tools & Ecosystem

Beginners typically start with scripting, then adopt orchestration tools as pipelines grow.

Pandas

Python scripting for small-scale ETL

Apache Airflow

Schedules & orchestrates pipelines

dbt

SQL-based transformation layer

Apache Kafka

Streaming data pipelines

Fivetran / Talend

Managed, no-code ETL connectors

Hands-on Demo: ETL on a Raw CSV

Standalone demo — load a raw CSV, clean and transform it, then write the output.

RAW INPUT

id	name	amount	date
1	A. Rao	1200	2026-01-04
2		-	2026/01/05
3	b khan	980	04-01-2026
4	C. Iyer	NaN	

Issues: missing name, inconsistent date formats, invalid amount values.

Not safe for direct use

TRANSFORMED OUTPUT

id	name	amount	date
1	A. Rao	1200	2026-01-04
2	Unknown	0	2026-01-05
3	B. Khan	980	2026-01-04
4	C. Iyer	0	2026-01-04

Clean names, standardized dates, validated numeric values — ready to load.

Ready for storage & analysis

ETL Demo: Code Walkthrough

How the raw-to-clean transformation happens in Python.

```
import pandas as pd

df = pd.read_csv('raw_orders.csv')

df['name'] = df['name'].fillna('Unknown')
df['name'] = df['name'].str.title()

df['date'] = pd.to_datetime(
    df['date'], errors='coerce'
).dt.strftime('%Y-%m-%d')

df['amount'] = pd.to_numeric(
    df['amount'], errors='coerce'
).fillna(0)

df.to_csv('clean_orders.csv', index=False)
```

STEP-BY-STEP

1. Load the raw CSV into a DataFrame
2. Fill missing names with a placeholder and standardize casing
3. Parse and reformat every date into one consistent format
4. Convert amount to numeric, replacing invalid values with 0
5. Write the cleaned data back out to a new CSV file

Data Cleaning Pipelines: Common Issues

The same handful of issues show up in nearly every raw dataset.

Missing Values

Gaps in records that skew analysis or break model inputs entirely.

Noise

Irrelevant, duplicated, or corrupted data that obscures real signal.

Duplicates

The same record entered multiple times, often with slight variations.

Outliers

Extreme values that distort averages and confuse models.

Data Cleaning: Missing Value Strategies

Deep dive — deciding what to do once you've found the gaps.

Handling Missing Values

There's no single correct fix for missing data — the right strategy depends on how much is missing and why.

- Drop: remove rows/columns when missingness is minimal and random
- Impute: fill with mean, median, mode, or a placeholder like ‘Unknown’
- Flag: add an indicator column so models can learn the pattern of missingness

EXAMPLE

70% of records are missing a ‘region’ field after a merge with a legacy system — flagging this before training prevents a regionally biased model.

Data Cleaning: Outliers & Noise

Deep dive — detecting and deciding what to do with extreme or corrupted values.

Detecting Outliers & Noise

Outliers can be genuine rare events or data entry errors — telling them apart is the real skill.

- Z-score: flag values several standard deviations from the mean
- IQR method: flag values far outside the interquartile range
- Always investigate before removing — a real event may be your most important signal

EXAMPLE

A customer age of “250” is clearly a data entry error, but a transaction of ₹10,00,000 among typical ₹1,000 purchases may be a genuine high-value sale.

Building a Repeatable Cleaning Pipeline

Repeatability turns cleaning from a one-off script into a trustworthy process.

Hands-on Demo: Data Cleaning Pipeline

Standalone demo — apply a repeatable pipeline to a batch of customer records.

BEFORE CLEANING

age	city	signup_date
250	mumbai	2026-01-04
-5	Delhi	04/01/2026
34	(blank)	2026-01-05
29	PUNE	2026-01-05

Issues: invalid ages, inconsistent casing, missing city, mixed date formats.

AFTER CLEANING

age	city	signup_date
NaN	Mumbai	2026-01-04
NaN	Delhi	2026-01-04
34	Unknown	2026-01-05
29	Pune	2026-01-05

Invalid ages flagged, city casing standardized, dates unified.

Cleaning Demo: Code Walkthrough

Applying validation rules consistently across a batch of records.

```
import pandas as pd
import numpy as np

df = pd.read_csv('customers_raw.csv')

# Validate age range
df.loc[
    (df['age'] < 0) | (df['age'] > 120),
    'age'
] = np.nan

# Standardize city casing
df['city'] = (
    df['city'].str.strip()
    .str.title()
    .fillna('Unknown')
)

# Unify date format
df['signup_date'] = pd.to_datetime(
    df['signup_date'], errors='coerce'
).dt.strftime('%Y-%m-%d')

df.to_csv('customers_clean.csv', index=False)
```

STEP-BY-STEP

1. Load the raw customer records
2. Replace impossible ages (negative or over 120) with NaN
3. Strip whitespace and standardize city name casing
4. Parse every signup date into one consistent format
5. Save the validated, cleaned file

RECAP

Part 1 Recap: Data Foundations & Cleaning

- Structured, semi-structured, and unstructured data each need different handling
- File format choice (CSV, JSON, PDF, DOCX, HTML) shapes how much preprocessing is needed
- ETL — Extract, Transform, Load — turns raw data into analysis-ready data
- A repeatable cleaning pipeline (profile → clean → validate → log) beats one-off scripts
- Hands-on: a raw CSV and a customer dataset were both cleaned through real pipelines

Text Preprocessing: Why It Matters

Raw text is messy — models need consistent, normalized input.

The same word can appear as “Running”, “running”, or “RUN” across a dataset — without preprocessing, a model may treat these as entirely unrelated tokens.

Text preprocessing standardizes raw language into a clean, consistent form before it ever reaches a model.

Raw Text → Model-Ready Text

"Running, RUNNING, and running!!" all become the same clean token after preprocessing.

Text Preprocessing: Tokenization

Splitting raw text into individual units a model can process.

Tokenization

Tokenization breaks a string of text into smaller units — usually words or sub-words — called tokens, the basic building blocks for every later NLP step.

- Word tokenization: split on spaces and punctuation
- Sub-word tokenization: splits rare words into meaningful fragments
- Sentence tokenization: splits a document into individual sentences

EXAMPLE

```
"AI systems need clean data." → ["AI", "systems", "need", "clean", "data", "."]
```

Text Preprocessing: Normalization & Case Folding

Reducing surface-level variation so the same meaning maps to the same text.

A

Normalization

Normalization standardizes text formatting — casing, punctuation, whitespace, and special characters — so identical meanings are represented identically.

- Case folding: converting all text to lowercase
- Removing punctuation, extra whitespace, and special characters
- Handling accents and unicode variations consistently

EXAMPLE

`"Café RESET!!  Password" → "cafe reset password"`

Text Preprocessing: Stopword Removal

Filtering out common words that carry little meaning on their own.

Stopword Removal

Stopwords (“the”, “is”, “at”, “and”) appear frequently but rarely help distinguish meaning — removing them can sharpen certain NLP tasks.

- Common in classic NLP pipelines: search, keyword extraction, topic modeling
- Modern transformer models often skip this step — they learn context directly
- Removing stopwords too aggressively can strip meaningful negation (e.g., “not”)

EXAMPLE

`"This is the best product I have used" → "best product used"`

Text Preprocessing: Stemming vs Lemmatization

Two ways to reduce words to a common base form.

Stemming

- Chops words down using simple rules
- Fast, but can produce non-words
- “Running”, “Runs” → “Run”
- “Better” → “Better” (unchanged)

Lemmatization

- Uses vocabulary & grammar to find the dictionary form
- Slower, but always produces a real word
- “Running”, “Runs” → “Run”
- “Better” → “Good”

Hands-on Demo: Text Preprocessing

Standalone demo — run raw customer feedback through a full preprocessing pipeline.

RAW TEXT

```
"The Support Team was AMAZING!!

I've NEVER had such quick replies... Will

definitely be recommending this to friends."
```

Mixed casing, punctuation, contractions, and stopwords throughout.

PREPROCESSED TOKENS

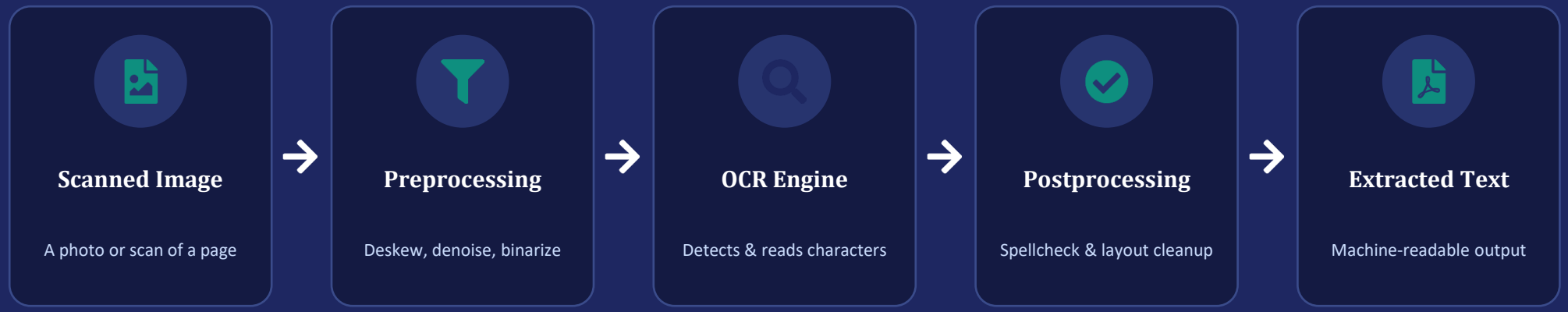
```
[
   'support', 'team', 'amazing',

    'never', 'quick', 'reply',

    'definitely', 'recommend', 'friend'
 ]
```

Lowercased, punctuation removed, stopwords dropped, lemmatized.

OCR Workflow Awareness: How OCR Works



This pipeline turns pixels into machine-readable, searchable text.

OCR Workflow Awareness: Common Pitfalls

OCR accuracy drops sharply outside of clean, high-quality scans.

Skewed Scans

Tilted pages confuse character detection and reading order.

Poor Contrast

Faded ink or low-resolution scans produce garbled output.

Handwriting

Standard OCR engines struggle badly with handwritten text.

Layout Loss

Tables and multi-column layouts often get flattened incorrectly.

OCR vs Native Text Extraction

Always check whether OCR is even necessary before running it.

Native Text Extraction

- For digitally-created PDFs/DOCX with embedded text
- Fast, accurate, no image processing needed
- Preserves exact original characters
- Always try this first

OCR

- For scanned documents or images with no embedded text
- Slower, and accuracy depends on scan quality
- Introduces potential character recognition errors
- Necessary only when there's no text layer to extract

Common OCR Tools & Libraries

A quick tour of the tools you'll encounter once OCR is actually needed.

Tesseract OCR

Free, open-source engine; the most widely used starting point

Google Cloud Vision

Managed cloud OCR API with strong layout detection

AWS Textract

Managed OCR with built-in table & form extraction

Azure AI Vision

Managed OCR API integrated with the Azure ecosystem

EasyOCR / PaddleOCR

Open-source Python libraries, strong multi-language support

Annotation Fundamentals: What & Why

Turning raw data into training examples a model can learn from.

What Is Annotation?

Annotation attaches ground-truth labels to raw data — the labels that supervised learning models are trained to predict.

- Without labels, most supervised models have nothing to learn from
- Annotation quality directly caps a model's achievable accuracy
- Can be done manually, semi-automatically, or via weak supervision

EXAMPLE

```
Text: "The delivery was delayed by a week."
Label: sentiment = negative
```

Annotation Fundamentals: Types of Annotation

The right annotation type depends entirely on the task.

Classification

One label per item, e.g. spam / not spam

Named Entity Recognition

Tagging spans like names, dates, places

Bounding Boxes

Marking object locations in images

Segmentation

Pixel-level outlines of objects

Transcription

Converting audio into labeled text

Annotation Fundamentals: Quality & Consistency

Inconsistent labeling teaches a model the wrong lessons.

Inconsistent Labeling

- Same kind of text labeled differently by different annotators
- No clear guidelines for edge cases
- Model learns conflicting signals
- Example: sarcasm labeled as both positive and negative

Consistent Labeling

- Clear labeling guidelines with edge-case examples
- Annotators trained on the same rubric
- Model learns a clean, reliable signal
- Example: a shared style guide resolves sarcasm consistently

Annotation: Inter-Annotator Agreement

Measuring how consistently different people label the same data.

Inter-Annotator Agreement (IAA)

IAA quantifies how often independent annotators agree on the same label — low agreement signals unclear guidelines or a genuinely ambiguous task.

- Common metric: Cohen's Kappa, which corrects for chance agreement
- Low IAA → revisit labeling guidelines before scaling annotation
- High IAA gives confidence that the labels reflect a real, learnable pattern

EXAMPLE

Two annotators label 100 reviews as positive/negative and agree on 92 — a high IAA, giving confidence in the label quality.

Common Data Annotation Tools

A quick tour of the tools teams use to label data at scale.

Label Studio

Free, open-source; supports text, image, audio & more

CVAT

Open-source, purpose-built for image & video annotation

Labelbox

Managed platform with workflow & QA tooling built in

Amazon SageMaker Ground Truth

Managed labeling with human review workforces

Prodigy

Scriptable, developer-first annotation tool for NLP

Data Quality Awareness

Poor data quality is the single biggest hidden cause of unreliable AI systems.

Missing Values

Gaps in records that skew analysis or break model inputs entirely.

Noise

Irrelevant, duplicated, or corrupted data that obscures real signal.

Bias

Data that over/under-represents groups, producing unfair AI outcomes.

Hallucination Risk

Sparse or low-quality context data leads GenAI to invent plausible-sounding falsehoods.

Deep Dive: Missing Values

Gaps in data are often invisible until a model fails downstream.

Why Values Go Missing

Missing data usually comes from optional form fields, failed system integrations, or merges between datasets that don't perfectly align.

- Detect: profile each column's missing-value percentage
- Fix: impute with a sensible default, or flag and exclude
- Avoid: silently converting missing values to zero — it hides the problem

EXAMPLE

70% of records are missing a ‘region’ field after a merge with a legacy system — flagging this before training prevents a regionally biased model.

Deep Dive: Noise & Duplicates

Redundant or corrupted data quietly dilutes real signal.

Where Noise Comes From

Noise creeps in through manual entry errors, duplicate imports, sensor glitches, and inconsistent formatting across sources.

- Detect: look for near-identical records with slight spelling differences
- Fix: standardize text casing/spacing, then deduplicate on a key field
- Avoid: deduplicating too aggressively and losing legitimate repeat records

EXAMPLE

“B. Khan” and “b khan” are the same customer entered twice — without standardization, they're counted as two different people.

Deep Dive: Bias in Data

Data can encode unfairness long before a model ever sees it.

How Bias Enters Data

Bias arises from sampling that over-represents certain groups, historical decisions baked into labels, or subjective human labeling.

- Detect: check representation across key segments (gender, region, age)
- Fix: rebalance samples, or use fairness-aware evaluation metrics
- Avoid: assuming a large dataset is automatically a representative one

EXAMPLE

A hiring dataset built from a company's past hires reflects historical hiring patterns — training on it directly can repeat past exclusions.

Deep Dive: Hallucination Risk

When AI systems confidently produce information that isn't true.

Why Models Hallucinate

Hallucination risk rises when the data grounding a GenAI response is sparse, outdated, or ambiguous — the model fills gaps with plausible guesses.

- Detect: monitor for confident answers with no supporting source
- Fix: ground responses in retrieval (RAG) over verified, current documents
- Avoid: relying on a model's internal knowledge for fast-changing facts

EXAMPLE

A support bot invents a refund policy that was deprecated 8 months ago because its retrieved context document was never updated.

Clean Data vs Poor-Quality Data

Poor-Quality Data

- Customer age recorded as “-5” and “250”
- Duplicate customer records with different spellings
- 70% of records missing the “region” field
- Support tickets skewed almost entirely to one language

Clean, Reliable Data

- Age values validated within a realistic range
- Records de-duplicated and standardized
- Missing fields flagged and imputed or excluded consistently
- Balanced representation checked across key segments

Data Quality Checklist

A practical checklist to run before any dataset feeds an AI system.

- [ ] Profile missing-value percentages across every column
- [ ] De-duplicate and standardize text fields (casing, spacing, spelling)
- [ ] Validate numeric and date ranges against realistic bounds
- [ ] Audit representation across key demographic or business segments
- [ ] Log data lineage: where did this data come from, and when was it last updated?

Dataset Versioning: Why Datasets Need Version Control

“Which data trained this model?” should always have a clear answer.

Why Version Datasets?

Datasets change over time — new rows, corrected labels, removed records. Without versioning, it becomes impossible to reproduce or debug a model's behavior.

- Enables reproducing a model's exact training conditions
- Makes it possible to roll back after a bad data update
- Supports auditing: proving what data a model was trained on

EXAMPLE

A model's accuracy suddenly drops after retraining — with dataset versioning, the team traces it to a corrupted label update in v1.4.

Dataset Versioning: Basic Practices

Simple habits that make datasets traceable and trustworthy.

Naming Conventions

e.g. dataset_v1.2_2026-01

DVC / Git-LFS

Version control for large data files

Checksums / Hashing

Detect unintended changes

Changelogs

Document what changed, and why

Dataset Registries

Central catalog of dataset versions

SQL Basics: Why SQL, Even in an AI World

GenAI hasn't replaced SQL — it depends on it.

- Most enterprise data still lives in structured, relational databases
- AI pipelines often combine SQL filtering with other retrieval methods
- Feature stores for ML models are commonly queried using SQL
- SQL is a universal skill — portable across almost every data platform

Anatomy of a SQL Query

Every query is built from the same handful of clauses, always in this order.

```
SELECT category, SUM(amount)
```
→ Choose columns / calculations

```
FROM sales
```
→ Choose the source table

```
WHERE quarter = 'Q4'
```
→ Filter individual rows

```
GROUP BY category
```
→ Aggregate rows into groups

```
ORDER BY SUM(amount) DESC
```
→ Sort the final results

```
LIMIT 5
```
→ Restrict the number of rows

SELECT & Column Selection

Choosing exactly which columns to retrieve from a table.

SELECT

Names the columns you want returned — or use * to return every column.

- SELECT name, age FROM customers; — returns only those two columns
- SELECT * FROM customers; — returns every column (use sparingly in production)
- Columns can be renamed on the fly using AS: SELECT name AS full_name

```
SELECT name, age, city
FROM customers;
```

Tip: selecting only the columns you need keeps queries faster and results easier to read.

WHERE & Filtering Conditions

Keeping only the rows that match a condition.

WHERE

Filters rows before any grouping happens, using comparison and logical operators.

- Comparisons: =, !=, >, <, >=, <=
- Logic: AND, OR, NOT to combine multiple conditions
- Pattern matching: LIKE '%mumbai%' for partial text matches

```
SELECT * FROM customers
WHERE city = 'Mumbai'
  AND age > 25;
```

WHERE filters individual rows — it runs before GROUP BY, not after.

JOIN: Combining Tables

Merging rows from two related tables using a shared key.

customers

customer_id | name

shared key

orders

customer_id | amount

JOIN result

name | amount

```
SELECT c.name, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

SQL JOIN Types

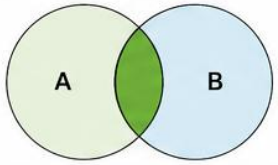
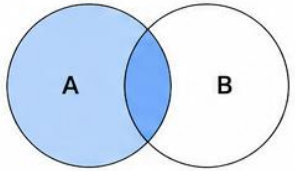
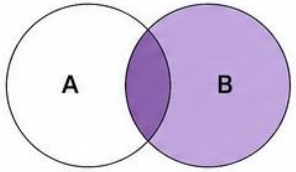
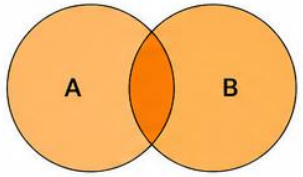
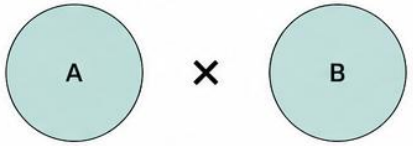
JOIN TYPE	VENN DIAGRAM	EXAMPLE RESULT	SQL SYNTAX
INNER JOIN		A.id | A.val | B.id | B.val 1 | A1 | 1 | B1 2 | A2 | 2 | B2	SELECT * FROM A INNER JOIN B ON A.id = B.id;
LEFT JOIN		A.id | A.val | B.id | B.val 1 | A1 | 1 | B1 2 | A2 | 2 | B2 3 | A3 | NULL | NULL	SELECT * FROM A LEFT JOIN B ON A.id = B.id;
RIGHT JOIN		A.id | A.val | B.id | B.val 1 | A1 | 1 | B1 2 | A2 | 2 | B2 NULL | NULL | 3 | B3	SELECT * FROM A RIGHT JOIN B ON A.id = B.id;
FULL OUTER JOIN		A.id | A.val | B.id | B.val 1 | A1 | 1 | B1 2 | A2 | 2 | B2 3 | A3 | NULL | NULL NULL | NULL | 4 | B4	SELECT * FROM A FULL OUTER JOIN B ON A.id = B.id;
CROSS JOIN		A.id | A.val | B.id | B.val 1 | A1 | 1 | B1 1 | A1 | 2 | B2 2 | A2 | 1 | B1 2 | A2 | 2 | B2	SELECT * FROM A CROSS JOIN B;

TABLE A

id	val
1	A1
2	A2
3	A3

TABLE B

id	val
1	B1
2	B2
4	B4

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN

NULL means no matching value.

GROUP BY & Aggregation Functions

Summarizing many rows into meaningful totals.

GROUP BY

Groups rows sharing the same value, so aggregate functions can summarize each group.

- COUNT() — number of rows in a group
- SUM() / AVG() — total or average of a numeric column
- MIN() / MAX() — smallest or largest value in a group

```
SELECT category, COUNT(*) AS orders,
       AVG(amount) AS avg_amount
FROM sales
GROUP BY category;
```

Every non-aggregated column in SELECT must also appear in GROUP BY.

HAVING vs WHERE

A common beginner confusion — they filter at different stages.

WHERE

- Filters individual rows
- Runs before grouping/aggregation
- Cannot reference aggregate functions
- Example: WHERE amount > 100

HAVING

- Filters grouped results
- Runs after GROUP BY aggregation
- Can reference aggregate functions
- Example: HAVING SUM(amount) > 10000

ORDER BY & LIMIT

Sorting results and restricting how many rows come back.

ORDER BY

Sorts the final result set — ascending (ASC, default) or descending (DESC).

- ORDER BY amount DESC — highest values first
- ORDER BY name ASC — alphabetical order
- LIMIT 5 — keep only the first N rows after sorting

```
SELECT name, amount FROM sales
ORDER BY amount DESC
LIMIT 5;
```

ORDER BY and LIMIT are almost always the last clauses in a query.

Subqueries: Queries Inside Queries

A query nested inside another — the inner query runs first.

OUTER QUERY · runs second, using the inner result

```
SELECT * FROM customers
WHERE customer_id IN (
```

INNER QUERY · runs first

```
SELECT customer_id FROM orders
WHERE amount > 1000
```

```
);
```

Finds all customers who have placed at least one order over 1000.

Common SQL Mistakes Beginners Make

Watch for these pitfalls — they trip up almost everyone at first.

No WHERE on UPDATE/DELETE

Forgetting a WHERE clause updates or deletes every row in the table.

= instead of IS NULL

NULL can't be compared with =; use IS NULL / IS NOT NULL instead.

Confusing HAVING & WHERE

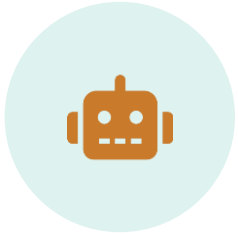
Using HAVING to filter rows, or WHERE to filter aggregates, causes errors.

Overusing SELECT *

Pulling every column wastes performance and hides intent in production code.

Why SQL Still Matters in AI Pipelines

Structured querying and AI retrieval work together, not against each other.



SQL + AI, Together

Modern AI systems often combine SQL for structured metadata with other retrieval methods for unstructured meaning — each covering what the other can't.

- RAG-style systems use SQL to filter documents by metadata before deeper retrieval
- Feature stores for ML models are queried with SQL for training data
- Text-to-SQL is itself a growing GenAI application category

EXAMPLE

A support-bot pipeline first uses SQL to filter documents to the customer's product line, then runs deeper retrieval only within that filtered set.

Hands-on Demo: Querying a Sample Dataset

Standalone demo — query a sample dataset directly to answer a real business question.

QUESTION: Which product category generated the most revenue last quarter?

```
SELECT category, SUM(amount) AS revenue
FROM sales
WHERE quarter = 'Q4'
GROUP BY category
ORDER BY revenue DESC;
```

RESULT

category	revenue
Electronics	₹ 18,42,300
Home & Kitchen	₹ 12,05,750
Fashion	₹ 9,88,420

SQL turns raw tables into direct, decision-ready answers — no manual spreadsheet work needed.

SQL Demo: Query Breakdown

Reading the revenue-by-category query clause by clause.

```
SELECT category,
       SUM(amount) AS revenue
FROM sales
WHERE quarter = 'Q4'
GROUP BY category
ORDER BY revenue DESC;
```

STEP-BY-STEP

1. SELECT: choose the category column plus a computed revenue total
2. FROM: read from the sales table
3. WHERE: keep only Q4 rows before any aggregation
4. GROUP BY: collapse rows into one per category
5. ORDER BY: sort categories from highest to lowest revenue

SQL Practice Challenge

Try writing the query yourself before revealing the answer.

CHALLENGE

Write a query to find the top 3 customers by total amount spent, across all their orders.

ANSWER

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 3;
```

PART 2 · DATA PRIVACY

Data Privacy Awareness: What Is PII

Personally Identifiable Information — the data that needs the most care.

Personally Identifiable Information

PII is any data that can identify a specific individual, either on its own or when combined with other data.

- Direct identifiers: name, email, phone number, government ID
- Indirect identifiers: date of birth + zip code can together identify someone
- AI pipelines must treat PII with extra caution at every stage: storage, processing, and output

EXAMPLE

A support ticket dataset containing customer emails, phone numbers, and billing addresses is full of PII — it cannot be used or shared carelessly.

Data Privacy: Anonymization Techniques

Reducing identifiability while keeping data useful for AI systems.

BEFORE ANONYMIZATION

name: Aditi Rao
email: aditi.rao@email.com
phone: 98765 43210
city: Mumbai

Contains multiple direct identifiers.

AFTER ANONYMIZATION

name: A*** R**
email: a****@email.com
phone: 98765 XXXXX
city: Mumbai

Masked fields retain usable structure without exposing identity.

PART 2 · DATA GOVERNANCE

Data Governance Fundamentals: Ownership & Access Control

Knowing who owns data, and who's allowed to touch it.

Ownership & Access Control

Data governance defines clear ownership for every dataset, and restricts access to only those roles that genuinely need it.

- Data owner: accountable for a dataset's quality and appropriate use
- Access control: role-based permissions (read, write, admin) per dataset
- Every access should be logged — supporting audits and incident response

EXAMPLE

A customer PII dataset is owned by the Data Privacy team; only specific, approved ML pipelines are granted read access, and every access is logged.

Data Governance: Compliance Basics

A beginner-safe overview of the concepts you'll encounter in industry.

GDPR / Data Laws

Regional regulations on personal data

Retention Policies

How long data can legally be kept

Audit Trails

Records of who accessed what, when

Consent Management

Tracking user permission for data use

RECAP

Key Takeaways

- Data-centric thinking means treating data quality and structure as core engineering work
- Structured, semi-structured, and unstructured data — and file formats like CSV, JSON, PDF, DOCX, HTML — each need different handling
- ETL and repeatable cleaning pipelines turn messy raw data into trustworthy input
- Text preprocessing, OCR awareness, and annotation quality directly shape model performance
- Dataset versioning, SQL, privacy, and governance keep AI systems reproducible, queryable, and responsible

Questions & Discussion

Thank you for joining — Data-Centric Thinking for AI System Development